

AMITY UNIVERSITY MAHARASHTRA
AMITY INSTITUTE OF INFORMATION TECHNOLOGY

CPP LAB FILE

MCA – SEM II



Name of the Student: **Narayan Niranjan Behera**

Program/Semester: **MCA / SEMESTER II**

Enrolment Number: **A710145025032**



AMITY UNIVERSITY MAHARASHTRA

Established vide Maharashtra Act No.13 of 2014, of Government of Maharashtra, and recognized under Section 2 (f) of UGC Act 1956.

Amity Institute of Information Technology

CERTIFICATE

This is to certify that, Mr. **Narayan Niranjana Behera**, Student of MCA Semester II (Batch 2025-27) has completed the C++ LAB JOURNAL during the academic year 2026-27, under the internal guidance of Prof. Dr. Manoj Devare .

Dr Manoj Devare
Guide

Dr Manoj Devare
HOI

Date:

INDEX

Practical No.	Practical Title	Pg. No.	Remarks
1.1	WAP to calculate factorial of a given number	5	
1.2	Menu driven arithmetic operations program	6	
2.1	Find largest number in an array	8	
2.2.1	Exchange contents using call by value	10	
2.2.2	Exchange contents using call by reference	12	
3.1.1	Area of geometrical figures using function overloading	13	
3.1.2	WAP to add two complex numbers using friend function.	15	
3.2.1	Student record management program	17	
3.2.2	Employee salary increment using array of objects	19	
3.2.3	Bank class with deposit and withdrawal operations	22	
4.1.1	Nested class student_info with date class	25	
4.1.2	Fibonacci series using copy constructor	28	
4.2.1	String comparison using operator overloading	30	
4.2.2	String concatenation using + operator	32	
4.2.3	Unary operator overloading for sign change	34	
5.1.1	Employee hierarchy using array of pointers	36	

5.1.2	Multiple inheritance using Employee & Qualification	40	
5.1.3	Multiple inheritance derives a class manager to implement this hierarchy.	44	
5.2.1	Student and Marks inheritance program	47	
5.2.2	File handling – write and read from file	50	
6.1	Write a C++ program to swap two numbers using pointers (call by reference using pointers).	52	
6.2	For swapping the values of two variables	54	
6.3	Write a C++ program to add two complex numbers using class and objects.	56	
6.4	Write a C++ program to demonstrate constructor overloading, destructor, and function overloading using a class Student.	58	
6.5	Write a C++ program to demonstrate friend function and friend class.	61	
6.6	function overloading for calculating area of rectangle, circle and triangle.	64	
6.7	Write a C++ program to swap two numbers using a temporary variable.	66	
7.1	Write a C++ program using command line arguments to perform different operations using switch-case.	67	
7.2	Write a C++ program to overload + operator to concatenate two strings.	70	
7.3	Write a C++ program to demonstrate single inheritance using classes Person and Student.	71	
7.4	Write a C++ program to store details of 5 books using array of objects and search a book by title.	74	

7.5	Write a C++ program to demonstrate function overloading by calculating the area of a circle and a rectangle.	78	
8.1	Create a class Employee with data members name, employee id, salary and designation. Take input from user and display employee details.	80	
8.2	Write a C++ program to find factorials of a number using a parameterized constructor.	82	
8.3	Demonstrate operator overloading (=) for copying one 2×2 matrix into another.	84	
8.4	Input a 3×3 matrix and find a row-wise sum.	87	
8.5	sort an array in descending order.	89	
8.6	find the transpose of a 3×3 matrix.	91	
9.1	Write a C++ program to add two numbers using a function and display the result	93	
9.2	Write characters into a file and then read and display them using file handling.	94	
9.3	Demonstrate constructors, destructor, and member functions using a class.	96	
9.4	Write a C++ program to copy contents of one file into another file using file handling.	99	
9.5	Write a C++ program to count the number of words in a file using file handling.	101	
9.6	Write a C++ program to store even numbers in one file and odd numbers in another file and display their contents.	102	
9.7	Write a C++ program to demonstrate structure, class, and pointer to object.	106	
9.8	Write a C++ program using switch-case to select gender based on input character (M/F).	108	

Practical 1.1

WAP to calculate factorial of a given number n.

```
#include <iostream>
using namespace std;

int main() {
    int n;
    long long factorial = 1;

    cout << "Enter a number: ";
    cin >> n;

    // Calculate factorial
    for(int i = 1; i <= n; i++) {
        factorial *= i;
    }

    cout << "Factorial of " << n << " = " << factorial;

    return 0;
}
```

Output

```
Narayan-MacBook-Air:module1 Narayan$ g++ factorial.cpp -o factorial
Narayan-MacBook-Air:module1 Narayan$ ./factorial
Enter a number: 6
Factorial of 6 = 720
Narayan-MacBook-Air:module1 Narayan$
```

Practical 1.2

WAP to do the following:

Generate the following menu:

Add two numbers.

Subtract two numbers.

Multiply two numbers.

Divide two numbers.

Exit.

Ask the user to input two integers and then input a choice from the menu.

Perform all the arithmetic operations which have been offered by the menu.

Checks for errors caused due to inappropriate entry by user and output a statement accordingly

```
#include <iostream>
using namespace std;

int main() {
    int num1, num2, choice;

    // Input two integers
    cout << "Enter first number: ";
    cin >> num1;

    cout << "Enter second number: ";
    cin >> num2;

    // Display menu
    cout << "\n===== MENU =====< endl;
    cout << "1. Add two numbers" << endl;
    cout << "2. Subtract two numbers" << endl;
    cout << "3. Multiply two numbers" << endl;
    cout << "4. Divide two numbers" << endl;
    cout << "5. Exit" << endl;

    cout << "\nEnter your choice: ";
    cin >> choice;

    // Perform operation based on choice
    switch(choice) {
        case 1:
            cout << "Sum = " << num1 + num2 << endl;
            break;
```

```

    case 2:
        cout << "Difference = " << num1 - num2 << endl;
        break;

    case 3:
        cout << "Product = " << num1 * num2 << endl;
        break;

    case 4:
        if(num2 != 0)
            cout << "Quotient = " << (float)num1 / num2 << endl;
        else
            cout << "Error: Division by zero is not allowed." << endl;
        break;

    case 5:
        cout << "Exiting program..." << endl;
        break;

    default:
        cout << "Error: Invalid choice entered." << endl;
}

return 0;
}

```

Output:

```

Narayan-MacBook-Air:module1 Narayan$ g++ menu1-2.cpp -o menu1-2
Narayan-MacBook-Air:module1 Narayan$ ./menu1-2
Enter first number: 55
Enter second number: 33

===== MENU =====
1. Add two numbers
2. Subtract two numbers
3. Multiply two numbers
4. Divide two numbers
5. Exit

Enter your choice: 3
Product = 1815
Narayan-MacBook-Air:module1 Narayan$

```


Practical 2.1

WAP to read a set of numbers in an array & to find the largest of them.

```
#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Enter the number of elements: ";
    cin >> n;

    int arr[n];

    // Input array elements
    cout << "Enter " << n << " numbers: ";
    for(int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    // Assume first element is largest
    int largest = arr[0];

    // Find largest element
    for(int i = 1; i < n; i++) {
        if(arr[i] > largest) {
            largest = arr[i];
        }
    }

    // Display result
    cout << "Largest number = " << largest << endl;

    return 0;
}
```

Output:

```
Narayan-MacBook-Air:module1 Narayan$ g++ largest-element-2-1.cpp -o
largest-element-2-1
Narayan-MacBook-Air:module1 Narayan$ ./largest-element-2-1
Enter the number of elements: 5
Enter 5 numbers: 2
3
4
5
6
Largest number = 6
Narayan-MacBook-Air:module1 Narayan$
```

Practical 2.2

1. WAP to exchange contents of two variables using call by value.

```
#include <iostream>
using namespace std;

// Function using call by value
void swapValue(int a, int b) {
    int temp;
    temp = a;
    a = b;
    b = temp;

    cout << "Inside function after swapping:" << endl;
    cout << "a = " << a << " , b = " << b << endl;
}

int main() {
    int x, y;

    cout << "Enter two numbers: ";
    cin >> x >> y;

    cout << "Before swapping:" << endl;
    cout << "x = " << x << " , y = " << y << endl;

    swapValue(x, y);

    cout << "After function call:" << endl;
    cout << "x = " << x << " , y = " << y << endl;

    return 0;
}
```

Output:

```
Narayan-MacBook-Air:module1 Narayan$ g++ call-by-value-2-2.cpp -o  
call-by-value-2-2  
Narayan-MacBook-Air:module1 Narayan$ ./call-by-value-2-2  
Enter two numbers: 5  
3  
Before swapping:  
x = 5 , y = 3  
Inside function after swapping:  
a = 3 , b = 5  
After function call:  
x = 5 , y = 3  
Narayan-MacBook-Air:module1 Narayan$
```

2. WAP to exchange contents of two variables using call by reference.

```
#include <iostream>
using namespace std;

// Function using call by reference
void swapReference(int &a, int &b) {
    int temp;
    temp = a;
    a = b;
    b = temp;
}

int main() {
    int x, y;

    cout << "Enter two numbers: ";
    cin >> x >> y;

    cout << "Before swapping:" << endl;
    cout << "x = " << x << " , y = " << y << endl;

    swapReference(x, y);

    cout << "After swapping:" << endl;
    cout << "x = " << x << " , y = " << y << endl;

    return 0;
}
```

Output:

```
Narayan-MacBook-Air:module1 Narayan$ g++ call-by-reference-2-2.cpp -o
call-by-reference-2-2
Narayan-MacBook-Air:module1 Narayan$ ./call-by-reference-2-2
Enter two numbers: 44
99
Before swapping:
x = 44 , y = 99
After swapping:
x = 99 , y = 44
Narayan-MacBook-Air:module1 Narayan$
```

Practical 3.1

Calculate area of different geometrical figures (circle, rectangle, square, triangle) using function overloading.

```
#include <iostream>
using namespace std;

class Area {
public:
    // Area of square
    void area(float side) {
        cout << "Area of Square = " << side * side << endl;
    }

    // Area of rectangle
    void area(float length, float breadth) {
        cout << "Area of Rectangle = " << length * breadth << endl;
    }

    // Area of circle
    void area(double radius) {
        cout << "Area of Circle = " << 3.14 * radius * radius << endl;
    }

    // Area of triangle
    void area(float base, float height, int) {
        cout << "Area of Triangle = " << 0.5 * base * height << endl;
    }
};

int main() {
    Area a;

    a.area(4.0f);           // Square
    a.area(5.0f, 3.0f);     // Rectangle
    a.area(2.5);            // Circle
    a.area(6.0f, 4.0f, 1);  // Triangle

    return 0;
}
```

Output:

```
Narayan-MacBook-Air:module-3 narayan$ g++ function-overloading-3-1.cpp -o  
function-overloading-3-1
```

```
Narayan-MacBook-Air:module-3 narayan$ ./function-overloading-3-1
```

```
Area of Square = 16
```

```
Area of Rectangle = 15
```

```
Area of Circle = 19.625
```

```
Area of Triangle = 12
```

```
Narayan-MacBook-Air:module-3 narayan$
```

WAP to add two complex numbers using friend function.

```
#include <iostream>
using namespace std;

class Complex {
    int real, imag;

public:
    void input() {
        cout << "Enter real and imaginary parts: ";
        cin >> real >> imag;
    }

    // Friend function declaration
    friend Complex addComplex(Complex c1, Complex c2);

    void display() {
        cout << real << " + " << imag << "i" << endl;
    }
};

// Friend function definition
Complex addComplex(Complex c1, Complex c2) {
    Complex temp;

    temp.real = c1.real + c2.real;
    temp.imag = c1.imag + c2.imag;

    return temp;
}

int main() {
    Complex c1, c2, result;

    cout << "Enter first complex number:" << endl;
    c1.input();

    cout << "Enter second complex number:" << endl;
    c2.input();

    result = addComplex(c1, c2);
```



```
    cout << "Sum of complex numbers = ";  
    result.display();  
  
    return 0;  
}
```

Output:

```
Narayan-MacBook-Air:module-3 narayan$ g++ friend-function-3-1.cpp -o  
friend-function-3-1  
Narayan-MacBook-Air:module-3 narayan$ ./friend-function-3-1  
Enter first complex number:  
Enter real and imaginary parts: 1 2  
Enter second complex number:  
Enter real and imaginary parts: 3 4  
Sum of complex numbers = 4 + 6i  
Narayan-MacBook-Air:module-3 narayan$
```

Practical 3.2

A. WAP to maintain the student record which contains Roll number, Name, Marks1, Marks2, Marks3 as data member and getdata(), display() and setdata() as member functions.

```
#include <iostream>
using namespace std;

class Student {
    int rollno;
    char name[50];
    float marks1, marks2, marks3;

public:
    void getdata() {
        cout << "Enter Roll Number: ";
        cin >> rollno;

        cout << "Enter Name: ";
        cin >> name;

        cout << "Enter Marks in 3 subjects: ";
        cin >> marks1 >> marks2 >> marks3;
    }

    void setdata(int r, const char n[], float m1, float m2, float m3) {
        rollno = r;

        int i = 0;
        while(n[i] != '\0') {
            name[i] = n[i];
            i++;
        }
        name[i] = '\0';

        marks1 = m1;
        marks2 = m2;
        marks3 = m3;
    }

    void display() {
```

```

        cout << "\nStudent Details" << endl;
        cout << "Roll Number: " << rollno << endl;
        cout << "Name: " << name << endl;
        cout << "Marks1: " << marks1 << endl;
        cout << "Marks2: " << marks2 << endl;
        cout << "Marks3: " << marks3 << endl;
    }
};

int main() {
    Student s;

    s.getdata();
    s.display();

    return 0;
}

```

Output:

```

Narayan-MacBook-Air:module-3 narayan$ g++ student-record-3-2.cpp -o
student-record-3-2
Narayan-MacBook-Air:module-3 narayan$ ./student-record-3-2
Enter Roll Number: 04
Enter Name: Narayan
Enter Marks in 3 subjects: 99 98 97

Student Details
Roll Number: 4
Name: Narayan
Marks1: 99
Marks2: 98
Marks3: 97
Narayan-MacBook-Air:module-3 narayan$

```

B. WAP to increment the employee salaries on the basis of there designation (Manager-5000, General Manager-10000, CEO-20000, worker-2000). Use employee name, id, designation and salary as data member and inc_sal as member function (Use array of object).

```
#include <iostream>
#include <string>
using namespace std;

class Employee {
    string name, designation;
    int id;
    float salary;

public:
    void getdata() {
        cout << "\nEnter Employee ID: ";
        cin >> id;

        cout << "Enter Employee Name: ";
        cin >> name;

        cout << "Enter Designation: ";
        cin >> designation;

        cout << "Enter Salary: ";
        cin >> salary;
    }

    void inc_sal() {
        if(designation == "Manager")
            salary += 5000;
        else if(designation == "GeneralManager")
            salary += 10000;
        else if(designation == "CEO")
            salary += 20000;
        else if(designation == "worker")
            salary += 2000;
        else
            cout << "Invalid Designation!" << endl;
    }
}
```

```

void display() {
    cout << "\nEmployee ID: " << id << endl;
    cout << "Name: " << name << endl;
    cout << "Designation: " << designation << endl;
    cout << "Updated Salary: " << salary << endl;
}
};

int main() {
    int n;

    cout << "Enter number of employees: ";
    cin >> n;

    Employee emp[n];

    for(int i = 0; i < n; i++) {
        emp[i].getdata();
        emp[i].inc_sal();
    }

    cout << "\n--- Employee Details After Salary Increment ---" << endl;

    for(int i = 0; i < n; i++) {
        emp[i].display();
    }

    return 0;
}

```

Output:

```
Narayan-MacBook-Air:module-3 narayan$ g++ employee-salary-3-2.cpp -o  
employee-salary-3-2
```

```
Narayan-MacBook-Air:module-3 narayan$ ./employee-salary-3-2
```

```
Enter number of employees: 2
```

```
Enter Employee ID: 1
```

```
Enter Employee Name: Narayan
```

```
Enter Designation: Manager
```

```
Enter Salary: 10000
```

```
Enter Employee ID: 2
```

```
Enter Employee Name: Niharika
```

```
Enter Designation: worker
```

```
Enter Salary: 2000
```

```
--- Employee Details After Salary Increment ---
```

```
Employee ID: 1
```

```
Name: Narayan
```

```
Designation: Manager
```

```
Updated Salary: 15000
```

```
Employee ID: 2
```

```
Name: Niharika
```

```
Designation: worker
```

```
Updated Salary: 4000
```

```
Narayan-MacBook-Air:module-3 narayan$
```

C. Write a class bank, containing data member: Name of Depositor, A/c type, Type of A/c, Balance amount. Member function: To assign initial value, To deposit an amount, to withdraw an amount after checking the balance (which should be greater than Rs. 500) , To display name & balance

```
#include <iostream>
#include <string>
using namespace std;

class Bank {
    string depositorName;
    string accType;
    string typeOfAcc;
    float balance;

public:
    // Assign initial values
    void assign() {
        cout << "Enter Depositor Name: ";
        cin >> depositorName;

        cout << "Enter Account Type: ";
        cin >> accType;

        cout << "Enter Type of Account: ";
        cin >> typeOfAcc;

        cout << "Enter Initial Balance: ";
        cin >> balance;
    }

    // Deposit amount
    void deposit() {
        float amount;

        cout << "Enter amount to deposit: ";
        cin >> amount;

        balance += amount;

        cout << "Amount Deposited Successfully." << endl;
```

```

}

// Withdraw amount
void withdraw() {
    float amount;

    cout << "Enter amount to withdraw: ";
    cin >> amount;

    if(balance - amount >= 500) {
        balance -= amount;
        cout << "Withdrawal Successful." << endl;
    }
    else {
        cout << "Insufficient balance! Minimum Rs. 500 should remain." << endl;
    }
}

// Display details
void display() {
    cout << "\nAccount Holder Name: " << depositorName << endl;
    cout << "Balance Amount: " << balance << endl;
}
};

int main() {
    Bank b;

    b.assign();
    b.deposit();
    b.withdraw();
    b.display();

    return 0;
}

```


Output

```
Narayan-MacBook-Air:module-3 narayan$ g++  
bank-account-operation-3-2.cpp -o bank-account-operation-3-2  
Narayan-MacBook-Air:module-3 narayan$ ./bank-account-operation-3-2  
Enter Depositor Name: Narayan  
Enter Account Type: Saving  
Enter Type of Account: Personal  
Enter Initial Balance: 100000  
Enter amount to deposit: 900000  
Amount Deposited Successfully.  
Enter amount to withdraw: 50000  
Withdrawal Successful.  
  
Account Holder Name: Narayan  
Balance Amount: 950000  
Narayan-MacBook-Air:module-3 narayan$
```

Practical 4.1

WAP to define nested class 'student_info' which contains data members such as name, roll number and sex and also consists of one more class 'date', whose data members are day, month and year. The data is to be read from the keyboard & displayed on the screen.

```
#include <iostream>
#include <string>
using namespace std;

class student_info {
private:
    string name;
    int rollno;
    char sex;

    // Nested class
    class date {
public:
        int day, month, year;

        void getDate() {
            cout << "Enter Day: ";
            cin >> day;

            cout << "Enter Month: ";
            cin >> month;

            cout << "Enter Year: ";
            cin >> year;
        }

        void displayDate() {
            cout << day << "/" << month << "/" << year << endl;
        }
    };

    date dob;
```

```

public:
    void getData() {
        cout << "Enter Name: ";
        cin >> name;

        cout << "Enter Roll Number: ";
        cin >> rollno;

        cout << "Enter Sex (M/F): ";
        cin >> sex;

        cout << "\nEnter Date of Birth:" << endl;
        dob.getDate();
    }

    void display() {
        cout << "\n--- Student Information ---" << endl;
        cout << "Name: " << name << endl;
        cout << "Roll Number: " << rollno << endl;
        cout << "Sex: " << sex << endl;

        cout << "Date of Birth: ";
        dob.displayDate();
    }
};

int main() {
    student_info s;

    s.getData();
    s.display();

    return 0;
}

```

Output:

```
Narayan-MacBook-Air:module-4 narayan$ g++ student_info-4-1.cpp -o student_info-4-1
```

```
Narayan-MacBook-Air:module-4 narayan$ ./student_info-4-1
```

```
Enter Name: Narayan
```

```
Enter Roll Number: 04
```

```
Enter Sex (M/F): M
```

```
Enter Date of Birth:
```

```
Enter Day: 16
```

```
Enter Month: 12
```

```
Enter Year: 2002
```

```
--- Student Information ---
```

```
Name: Narayan
```

```
Roll Number: 4
```

```
Sex: M
```

```
Date of Birth: 16/12/2002
```

```
Narayan-MacBook-Air:module-4 narayan$
```

WAP to generate a series of Fibonacci numbers using copy constructor, where it is defined outside the class using scope resolution operator.

```
#include <iostream>
using namespace std;

class Fibonacci {
    int a, b;

public:
    // Constructor
    Fibonacci(int x, int y) {
        a = x;
        b = y;
    }

    // Copy Constructor Declaration
    Fibonacci(Fibonacci &f);

    void display(int n);
};

// Copy Constructor defined outside the class
Fibonacci::Fibonacci(Fibonacci &f) {
    a = f.a;
    b = f.b;
}

// Function to display Fibonacci series
void Fibonacci::display(int n) {
    int next;

    cout << a << " " << b << " ";

    for(int i = 3; i <= n; i++) {
        next = a + b;
        cout << next << " ";

        a = b;
        b = next;
    }
}
```

```

int main() {
    int n;

    cout << "Enter number of terms: ";
    cin >> n;

    Fibonacci f1(0, 1);

    // Using copy constructor
    Fibonacci f2 = f1;

    cout << "Fibonacci Series: ";
    f2.display(n);

    return 0;
}

```

Output:

```

Narayan-MacBook-Air:module-4 narayan$ g++ fibonacci_series-4-1.cpp -o
fibonacci_series-4-1
Narayan-MacBook-Air:module-4 narayan$ ./fibonacci_series-4-1
Enter number of terms: 10
Fibonacci Series: 0 1 1 2 3 5 8 13 21 34
Narayan-MacBook-Air:module-4 narayan$

```

Practical 4.2

Write a class string to compare two strings, overload (=) operator.

```
#include <iostream>
#include <string>
using namespace std;

class StringCompare {
    string str;

public:
    void getString() {
        cout << "Enter String: ";
        cin >> str;
    }

    // Overloading == operator
    bool operator == (StringCompare s) {
        return (str == s.str);
    }
};

int main() {
    StringCompare s1, s2;

    cout << "Enter First String" << endl;
    s1.getString();

    cout << "Enter Second String" << endl;
    s2.getString();

    if(s1 == s2)
        cout << "Both strings are equal." << endl;
    else
        cout << "Strings are not equal." << endl;

    return 0;
}
```

Output:

```
Narayan-MacBook-Air:module-4 narayan$ g++ compare_two_string-4-2.cpp  
-o compare_two_string-4-2  
Narayan-MacBook-Air:module-4 narayan$ ./compare_two_string-4-2  
Enter First String  
Enter String: Hello  
Enter Second String  
Enter String: Hello  
Both strings are equal.  
Narayan-MacBook-Air:module-4 narayan$
```


Write a class to concatenate two strings, overload (+) operator.

```
#include <iostream>
#include <string>
using namespace std;

class StringConcat {
    string str;

public:
    void getString() {
        cout << "Enter String: ";
        cin >> str;
    }

    // Overloading + operator
    StringConcat operator + (StringConcat s) {
        StringConcat temp;
        temp.str = str + s.str;
        return temp;
    }

    void display() {
        cout << "Concatenated String = " << str << endl;
    }
};

int main() {
    StringConcat s1, s2, s3;

    cout << "Enter First String" << endl;
    s1.getString();

    cout << "Enter Second String" << endl;
    s2.getString();

    s3 = s1 + s2;

    s3.display();
    return 0;
}
```

Output:

```
Narayan-MacBook-Air:module-4 narayan$ g++  
concatenate_two_string-4-2.cpp -o concatenate_two_string-4-2  
Narayan-MacBook-Air:module-4 narayan$ ./concatenate_two_string-4-2  
Enter First String  
Enter String: Good  
Enter Second String  
Enter String: Morning  
Concatenated String = GoodMorning  
Narayan-MacBook-Air:module-4 narayan$
```

Create a class item, having two data members x & y, overload '-'(unary operator) to change the sign of x and y.

```
#include <iostream>
using namespace std;

class Item {
    int x, y;

public:
    void getData() {
        cout << "Enter values of x and y: ";
        cin >> x >> y;
    }

    // Overloading unary - operator
    void operator - () {
        x = -x;
        y = -y;
    }

    void display() {
        cout << "x = " << x << endl;
        cout << "y = " << y << endl;
    }
};

int main() {
    Item obj;

    obj.getData();

    cout << "\nBefore changing sign:" << endl;
    obj.display();

    -obj;
    cout << "\nAfter changing sign:" << endl;
    obj.display();

    return 0;
}
```

Output:

```
Narayan-MacBook-Air:module-4 narayan$ g++ change-sign-4-2.cpp -o  
change-sign-4-2
```

```
Narayan-MacBook-Air:module-4 narayan$ ./change-sign-4-2
```

```
Enter values of x and y: 10 20
```

```
Before changing sign:
```

```
x = 10
```

```
y = 20
```

```
After changing sign:
```

```
x = -10
```

```
y = -20
```

```
Narayan-MacBook-Air:module-4 narayan$
```

Practical 5.1

Create a class Employee. Derive 3 classes from this class namely Programmer, Analyst and Project Leader. Take suitable attributes and operations. Write a C++ program to implement this using array of pointers

```
#include <iostream>
#include <string>
using namespace std;

// Base Class
class Employee {
protected:
    int empld;
    string name;
    float salary;

public:
    Employee(int id, string n, float s) {
        empld = id;
        name = n;
        salary = s;
    }

    virtual void display() {
        cout << "\nEmployee ID : " << empld;
        cout << "\nName      : " << name;
        cout << "\nSalary    : " << salary << endl;
    }

    virtual ~Employee() {}
};

// Derived Class Programmer
class Programmer : public Employee {
    string language;

public:
    Programmer(int id, string n, float s, string lang)
        : Employee(id, n, s) {
        language = lang;
    }
};
```

```

    }

    void display() override {
        cout << "\n----- Programmer -----";
        Employee::display();
        cout << "Language    : " << language << endl;
    }
};

```

// Derived Class Analyst

```

class Analyst : public Employee {
    string domain;

public:
    Analyst(int id, string n, float s, string d)
        : Employee(id, n, s) {
        domain = d;
    }

    void display() override {
        cout << "\n----- Analyst -----";
        Employee::display();
        cout << "Domain      : " << domain << endl;
    }
};

```

// Derived Class Project Leader

```

class ProjectLeader : public Employee {
    int teamSize;

public:
    ProjectLeader(int id, string n, float s, int t)
        : Employee(id, n, s) {
        teamSize = t;
    }

    void display() override {
        cout << "\n----- Project Leader -----";
        Employee::display();
        cout << "Team Size   : " << teamSize << endl;
    }
}

```

```
};

int main() {

    // Array of Base Class Pointers
    Employee* emp[3];

    emp[0] = new Programmer(101, "Rahul", 50000, "C++");
    emp[1] = new Analyst(102, "Sneha", 60000, "Data Analysis");
    emp[2] = new ProjectLeader(103, "Amit", 80000, 10);

    for (int i = 0; i < 3; i++) {
        emp[i]->display();
    }

    // Free Memory
    for (int i = 0; i < 3; i++) {
        delete emp[i];
    }

    return 0;
}
```

Output:

```
Narayan-MacBook-Air:module-5 narayan$ g++ programA-5-1.cpp -o  
programA-5-1
```

```
Narayan-MacBook-Air:module-5 narayan$ ./programA-5-1
```

```
----- Programmer -----
```

```
Employee ID : 101
```

```
Name       : Rahul
```

```
Salary      : 50000
```

```
Language    : C++
```

```
----- Analyst -----
```

```
Employee ID : 102
```

```
Name       : Sneha
```

```
Salary      : 60000
```

```
Domain      : Data Analysis
```

```
----- Project Leader -----
```

```
Employee ID : 103
```

```
Name       : Amit
```

```
Salary      : 80000
```

```
Team Size   : 10
```

```
Narayan-MacBook-Air:module-5 narayan$
```


Create two classes namely Employee and Qualification. Using multiple inheritance derive a class Scientist. Take suitable attributes and operations. Write a C++ program to implement this hierarchy.

```
#include <iostream>
#include <string>
using namespace std;

// Base Class Employee
class Employee {
protected:
    int empld;
    string name;

public:
    Employee(int id, string n) {
        empld = id;
        name = n;
    }

    void showEmployee() {
        cout << "\nEmployee ID : " << empld;
        cout << "\nName      : " << name;
    }
};

// Base Class Qualification
class Qualification {
protected:
```

```

    string degree;
    string university;

public:
    Qualification(string d, string u) {
        degree = d;
        university = u;
    }

    void showQualification() {
        cout << "\nDegree    : " << degree;
        cout << "\nUniversity : " << university;
    }
};

// Derived Class Scientist
class Scientist : public Employee, public Qualification {
    string researchField;

public:
    Scientist(int id, string n,
              string d, string u,
              string rf)
        : Employee(id, n), Qualification(d, u) {

        researchField = rf;
    }

    void display() {

```

```

        cout << "\n===== Scientist Details =====";
        showEmployee();
        showQualification();
        cout << "\nResearch    : " << researchField << endl;
    }
};

```

```

int main() {

    Scientist s1(201, "Dr. Mehta",
                 "PhD Physics",
                 "IIT Bombay",
                 "Quantum Mechanics");

    s1.display();

    return 0;
}

```

Output:

```
Narayan-MacBook-Air:module-5 narayan$ g++ programB-5-1.cpp -o programB-5-1
```

```
Narayan-MacBook-Air:module-5 narayan$ ./programB-5-1
```

```
===== Scientist Details =====
```

```
Employee ID : 201
```

```
Name       : Dr. Mehta
```

```
Degree     : PhD Physics
```

```
University : IIT Bombay
```

```
Research   : Quantum Mechanics
```

```
Narayan-MacBook-Air:module-5 narayan$
```

Create two classes namely Employee and Qualification. Using multiple inheritance derives a class Manager. Take suitable attributes and operations. Write a C++ program to implement this hierarchy.

```
#include <iostream>
#include <string>
using namespace std;

// Base Class Employee
class Employee {
protected:
    int empld;
    string name;

public:
    Employee(int id, string n) {
        empld = id;
        name = n;
    }

    void showEmployee() {
        cout << "\nEmployee ID : " << empld;
        cout << "\nName      : " << name;
    }
};

// Base Class Qualification
class Qualification {
protected:
    string degree;
    string university;

public:
    Qualification(string d, string u) {
        degree = d;
        university = u;
    }

    void showQualification() {
        cout << "\nDegree      : " << degree;
        cout << "\nUniversity : " << university;
    }
};
```

```

    }
};

// Derived Class Manager
class Manager : public Employee, public Qualification {
    string department;

public:
    Manager(int id, string n,
            string d, string u,
            string dept)
        : Employee(id, n), Qualification(d, u) {

        department = dept;
    }

    void display() {
        cout << "\n==== Manager Details =====";
        showEmployee();
        showQualification();
        cout << "\nDepartment : " << department << endl;
    }
};

int main() {

    Manager m1(301, "Karan",
               "MBA",
               "Delhi University",
               "HR");

    m1.display();

    return 0;
}

```

Output:

```
Narayan-MacBook-Air:module-5 narayan$ g++ programC-5-1.cpp -o  
programC-5-1
```

```
Narayan-MacBook-Air:module-5 narayan$ ./programC-5-1
```

```
===== Manager Details =====
```

```
Employee ID : 301
```

```
Name       : Karan
```

```
Degree     : MBA
```

```
University : Delhi University
```

```
Department : HR
```

```
Narayan-MacBook-Air:module-5 narayan$
```

Practical 5.2

WAP to create a class Student containing Name & Class as parameters. Create another class Marks which inherits Student class taking own data members as mark1 & mark2 and show result.

```
#include <iostream>
#include <string>
using namespace std;

// Base Class
class Student {
protected:
    string name;
    string studentClass;

public:
    void getStudentData() {
        cout << "Enter Student Name : ";
        cin >> name;

        cout << "Enter Class : ";
        cin >> studentClass;
    }
};

// Derived Class
class Marks : public Student {
private:
    int mark1, mark2, total;
    float percentage;

public:
    void getMarks() {
        cout << "Enter Mark 1 : ";
        cin >> mark1;

        cout << "Enter Mark 2 : ";
        cin >> mark2;
    }
}
```



```

void calculate() {
    total = mark1 + mark2;
    percentage = total / 2.0;
}

void display() {
    cout << "\n===== Student Result =====";
    cout << "\nName      : " << name;
    cout << "\nClass     : " << studentClass;
    cout << "\nMark 1    : " << mark1;
    cout << "\nMark 2    : " << mark2;
    cout << "\nTotal     : " << total;
    cout << "\nPercentage : " << percentage << "%" << endl;
}
};

int main() {

    Marks m;

    m.getStudentData();
    m.getMarks();

    m.calculate();
    m.display();

    return 0;
}

```

Output:

```
Narayan-MacBook-Air:module-5 narayan$ g++ programA-5-2.cpp -o  
programA-5-2
```

```
Narayan-MacBook-Air:module-5 narayan$ ./programA-5-2
```

```
Enter Student Name : Narayan
```

```
Enter Class : MCA
```

```
Enter Mark 1 : 99
```

```
Enter Mark 2 : 98
```

```
===== Student Result =====
```

```
Name      : Narayan
```

```
Class     : MCA
```

```
Mark 1    : 99
```

```
Mark 2    : 98
```

```
Total    : 197
```

```
Percentage : 98.5%
```

```
Narayan-MacBook-Air:module-5 narayan$
```

WAP to read data from keyboard and write it to a file. After writing is completed, the file is closed. The program again opens the same file and reads it.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {

    string data;

    // Writing to File
    ofstream fout("sample.txt");

    cout << "Enter data to write into file : ";
    getline(cin, data);

    fout << data;

    fout.close();

    cout << "\nData written to file successfully.";

    // Reading from File
    ifstream fin("sample.txt");

    string readData;

    getline(fin, readData);

    cout << "\n\nData read from file : ";
    cout << readData << endl;

    fin.close();

    return 0;
}
```

Output:

```
Narayan-MacBook-Air:module-5 narayan$ g++ programB-5-2.cpp -o  
programB-5-2
```

```
Narayan-MacBook-Air:module-5 narayan$ ./programB-5-2
```

```
Enter data to write into file : The is testing for creation of txt file using c++.
```

```
Data written to file successfully.
```

```
Data read from file : The is testing for creation of txt file using c++.
```

```
Narayan-MacBook-Air:module-5 narayan$
```

Module-1

Practical 6.1: Write a C++ program to swap two numbers using pointers (call by reference using pointers).

```
#include<iostream>
using namespace std;

void swap(int *x, int *y){
    int temp = *x;
    *x = *y;
    *y = temp;

    cout << "x = " << *x << endl;
    cout << "y = " << *y << endl;
}

int main(){
    int a = 10, b = 20;
    cout << "Before calling swap function" << endl;
    cout << "a = " << a << "\nb = " << b << endl;

    swap(&a, &b); // call by pointers
    cout << "After calling swap function" << endl;
    cout << "a = " << a << "\nb = " << b << endl;

    return 0;
}
```

Output

→ module1 git:(module-1) ✗ g++ call-by-pointers.cpp -o call-by-pointers

→ module1 git:(module-1) ✗ ./call-by-pointers

Before calling swap function

a = 10

b = 20

x = 20

y = 10

After calling swap function

a = 20

b = 10

→ module1 git:(module-1) ✗

Practical 6.2: For swapping the values of two variables

```
#include <iostream>
using namespace std;

void swap(int &m, int &n){
    cout << "The values of m and n before swapping- within function" << endl;
    cout << "m = " << m << endl << "n = " << n << endl;

    int temp;
    temp = m;
    m = n;
    n = temp;

    cout << "The values of m and n after swapping - Within Function" << endl;
    cout << "m = " << m << endl << "n = " << n << endl;
}

int main(){
    int x, y;
    x = 10;
    y = 20;

    cout << "The values of x and y before swapping" << endl;
    cout << "x = " << x << endl << "y = " << endl;
    swap(x, y);

    cout << "The values of x and y after swapping" << endl;
    cout << "x = " << x << endl << "y = " << y << endl;

    return 0;
}
```

Output

```
module1 git:(main) ✗ g++ call-by-reference.cpp -o call-by-reference
➔ module1 git:(main) ✗ ./call-by-reference
The values of x and y before swapping
x = 10
y =
The values of m and n before swapping- within function
m = 10
n = 20
The values of m and n after swapping - Within Function
m = 20
n = 10
The values of x and y after swapping
x = 20
y = 10
➔ module1 git:(main) ✗
*/
```


Practical 6.3: Write a C++ program to add two complex numbers using class and objects.

```
#include<iostream>
using namespace std;

class Complex{
    int a, b;
public:
    Complex(int x, int y){
        a = x;
        b = y;
    }

    void display(){
        cout << endl << a << "i + " << b << endl;
    }

    Complex add(Complex p, Complex q){
        p.a = p.a + q.a;
        p.b = p.b + q.b;
        return(p);
    }
};

int main(){
    Complex obj1(5, 8);
    Complex obj2(2, 3);

    cout << "The first complex number is" << endl;
    obj1.display();
    cout << endl;
    obj2.display();

    obj1 = obj1.add(obj1, obj2);
    cout << "\n After summation" << endl;
    obj1.display();

    return 0;
}
```

Output:

→ module1 git:(module-1) ✗ g++ complex-number-addition.cpp -o complex-number-addition

→ module1 git:(module-1) ✗ ./complex-number-addition

The first complex number is

$5i + 8$

$2i + 3$

After summation

$7i + 11$

→ module1 git:(module-1) ✗

Practical 6.4: Write a C++ program to demonstrate constructor overloading, destructor, and function overloading using a class Student.

```
#include <iostream>
#include <cstring>

using namespace std;

class Student
{
private:
    int roll;
    char name[30];
    int marks1, marks2;

public:
    Student();           // default constructor
    Student(int, int);   // parameterized constructor
    ~Student();          // destructor

    int add(int, int);
    int add();
};

int Student::add()
{
    return (marks1 + marks2);
}

int Student::add(int a, int b)
{
    marks1 = a;
    marks2 = b;
    return (marks1 + marks2);
}

// Default constructor
Student::Student()
{
    roll = 1;
    strcpy(name, "amity_student");

    cout << "The constructor is invoked" << endl;
```

```

    cout << "Roll = " << roll << endl;
    cout << "Name = " << name << endl;
}

// Destructor
Student::~Student()
{
    cout << "\nDestructor invoked!" << endl;
}

// Parameterized constructor
Student::Student(int x, int y)
{
    cout << "\nThe Parameterized constructor is invoked" << endl;

    marks1 = x;
    marks2 = y;

    cout << "The marks are =" << endl;
    cout << marks1 << endl;
    cout << marks2 << endl;
}

int main()
{
    Student obj(60, 80);

    int total;

    cout << "\nThe total after parameterized constructor is called" << endl;
    cout << obj.add() << endl;

    total = obj.add(67, 87);

    cout << "\nThe total = " << total << endl;

    return 0;
}

```

Output:

```
→ module1 git:(module-1) ✗ g++ constructors.cpp -o constructors  
→ module1 git:(module-1) ✗ ./constructors
```

The Parameterized constructor is invoked

The marks are =

60

80

The total after parameterized constructor is called

140

The total = 154

Destructor invoked!

```
→ module1 git:(module-1) ✗
```

Practical 6.5: Write a C++ program to demonstrate friend function and friend class.

```
#include <iostream>
#include<cstring>
using namespace std;

class B;

class Student{
private:
    int marks1;
    int marks2;
    int total;
    char name[10];

public:
    friend int main(); // friend function
    friend class B; // friend class

    void read();
    void display();
};

class B{
public:
    void getSData(Student s1){
        cout << endl << "The Student name = " << s1.name << endl;
    }
};

void Student::read(){
    cout << endl << "Enter student name" << endl;
    cin >> name;

    cout << "Enter marks of subject 1" << endl;
    cin >> marks1;

    cout << "Enter marks of subject 2" << endl;
    cin >> marks2;

    total = marks1 + marks2;
}
```

```

void Student::display(){
    cout << endl << "Student Name = " << name;
    cout << endl << "\tMarks 1 = " << marks1;
    cout << endl << "\tMarks 2 = " << marks2;
    cout << endl << "\tTotal marks = " << total << endl;
}

int main(){
    Student obj;

    // allowed because main is friend
    obj.marks1 = 50;

    cout << endl << "The marks are " << obj.marks1 << endl;

    obj.read();
    obj.display();

    cout << endl << "Calling class b function" << endl;

    B Objb;
    Objb.getSData(obj);

    return 0;
}

```

Output:

```
➔ module1 git:(module-1) X g++ friend-function.cpp -o friend-function
➔ module1 git:(module-1) X ./friend-function
```

The marks are 50

Enter student name

Narayan

Enter marks of subject 1

99

Enter marks of subject 2

98

Student Name = Narayan

Marks 1 = 99

Marks 2 = 98

Total marks = 197

Calling class b function

The Student name = Narayan

```
➔ module1 git:(module-1) X
```


Practical 6.6: Write a C++ program to demonstrate function overloading for calculating area of rectangle, circle and triangle.

```
#include<iostream>
using namespace std;

class A{
private:
    int h, w;
    float base, h1;
    float r;

public:
    // Area of rectangle
    inline int area(int height, int width){
        h = height;
        w = width;
        return (h * w);
    }

    // area of circle
    float area(float radius){
        r = radius;
        return (3.14f * r * r);
    }

    // area of triangle
    float area (float b, float h){
        base = b;
        h1 = h;
        return (0.5f * base * h1);
    }
};

int main(){
    A obj;

    cout << "\nThe Area of Rectangle = " << obj.area(10, 20) << endl;

    cout << "The Area of Circle = " << obj.area(4.5f) << endl;

    cout << "The Area of Triangle = " << obj.area(6.0f, 7.0f) << endl;
```

```
    return 0;  
}
```

Output:

→ module1 git:(module-1) ✗ g++ function-overloading.cpp -o function-overloading

→ module1 git:(module-1) ✗ ./function-overloading

The Area of Rectangle = 200

The Area of Circle = 63.585

The Area of Triangle = 21

→ module1 git:(module-1) ✗

Practical 1.7: Write a C++ program to swap two numbers using a temporary variable.

```
#include<iostream>
using namespace std;

int main(){
    int m, n;
    cout << "Enter two numbers" << endl;
    cin >> m;
    cin >> n;

    cout << "Values before swapping" << endl;
    cout << "m = " << m << "\nn = " << n << endl;

    int temp = m;
    m = n;
    n = temp;

    cout << "After swapping the values are " << endl;
    cout << "m = " << m << "\nn = " << n << endl;

    return 0;
}
```

Output:

```
→ module1 git:(module-1) X g++ swapping-of-two-variables.cpp -o
swapping-of-two-variables
→ module1 git:(module-1) X ./swapping-of-two-variables
Enter two numbers
5
6
Values before swapping
m = 5
n = 6
After swapping the values are
m = 6
n = 5
→ module1 git:(module-1) X
```

Module 2

Practical 7.1: Write a C++ program using command line arguments to perform different operations using switch-case.

```
#include <iostream>
#include <cstdlib>
#include <cstring>

using namespace std;

int main(int argc, char* argv[]){
    // check if argument is provided
    if (argc < 2){
        cout << "Usage: program_name <option> " << endl;
        return 0;
    }

    char dest[5];
    strcpy(dest, argv[1]);

    cout << "Option = " << dest << endl;

    switch ((dest[0]))
    {
    case 'f':
        system("ls");
        break;

    case 'c':
        system("ls *.cpp");
        break;

    case 'd':
        system("rm -f matrix.out");
        cout << "matrix.exe deleted.";
        break;

    case 'h':
        cout << "\nUse f for listing all files" << endl;
        cout << "Example : program_name f" << endl;
```

```
    cout << "\nUse c for listing only cpp files" << endl;
    cout << "Example : program_name c" << endl;

    cout << "\nUse d for deleting matrix.exe" << endl;
    cout << "Example : program_name c" << endl;

    cout << "\nUse h for help" << endl;
    cout << "Example : program_name h" << endl;
    break;

default:
    cout << "\nWrong option!!" << endl;

}
return 0;
}
```

Output:

```
Narayan-MacBook-Air:module1 Narayan$ g++
1-main-function-with-arguments.cpp -o 1-main-function-with-arguments
Narayan-MacBook-Air:module1 Narayan$ ./1-main-function-with-arguments f
Option = f
1-main-function-with-arguments
1-main-function-with-arguments.cpp
Narayan-MacBook-Air:module1 Narayan$ ./1-main-function-with-arguments c
Option = c
1-main-function-with-arguments.cpp
Narayan-MacBook-Air:module1 Narayan$ ./1-main-function-with-arguments d
Option = d
Narayan-MacBook-Air:module1 Narayan$
Narayan-MacBook-Air:module1 Narayan$ ./1-main-function-with-arguments h
Option = h
```

Use f for listing all files

Example : program_name f

Use c for listing only cpp files

Example : program_name c

Use d for deleting matrix.exe

Example : program_name c

Use h for help

Example : program_name h

Narayan-MacBook-Air:module1 Narayan\$

Practical 7.2: Write a C++ program to overload + operator to concatenate two strings.

```
#include <iostream>
#include <cstring>

using namespace std;

class Connect{
private:
    char str[50];

public:
    // Constructor
    Connect(const char *s){
        strcpy(str, s);
    }

    // operator overloading
    void operator +(Connect obj){
        strcat(str, obj.str);

        cout << "The concatenated string = " << str << endl;
    }
};

int main(){
    Connect obj1("Amity ");
    Connect obj2("University");

    obj1 + obj2;

    return 0;
}
```

Output:

```
Narayan-MacBook-Air:module1 Narayan$ g++ 2-operator-overloading.cpp -o
2-operator-overloading
Narayan-MacBook-Air:module1 Narayan$ ./2-operator-overloading
The concatenated string = Amity University
Narayan-MacBook-Air:module1 Narayan$
```

Practical 7.3 Write a C++ program to demonstrate single inheritance using classes Person and Student.

```
#include <iostream>
#include<cstring>

using namespace std;

// base class
class Person{
private:
    char name[20];
    int age;

public:
    void setData(){
        strcpy(name, "Narayan");
        age = 23;
    }

    void printData(){
        cout << "Name = " << name << endl;
        cout << "Age = " << age << endl;
    }
};

// derived class
class Student : public Person{
private:
    char program[20];
    int enrol;

public:
    void setStudent(){
        strcpy(program, "MCA");
        enrol = 4;
    }

    void printStudent(){
        cout << "Enrollment Number = " << enrol << endl;
        cout << "Program = " << program << endl;
    }
};
```



```
int main(){
    Student obj;

    obj.setData();
    obj.setStudent();

    obj.printData();
    obj.printStudent();

    return 0;
}
```

Output:

```
Narayan-MacBook-Air:module1 Narayan$ g++ 3-simple-inheritance.cpp -o
3-simple-inheritance
Narayan-MacBook-Air:module1 Narayan$ ./3-simple-inheritance
Name = Narayan
Age = 23
Enrollment Number = 4
Program = MCA
Narayan-MacBook-Air:module1 Narayan$
```

Practical 7.4: Write a C++ program to store details of 5 books using array of objects and search a book by title.

```
#include <iostream>
#include <cstring>

using namespace std;

class Book
{
private:
    char title[30];
    char author[30];
    float cost;
    int accession_num;

public:

    void readData()
    {
        cout << "Enter Book Title: ";
        cin.getline(title, 30);

        cout << "Enter Author Name: ";
        cin.getline(author, 30);

        cout << "Enter Accession Number: ";
        cin >> accession_num;

        cout << "Enter Cost: ";
        cin >> cost;

        cin.ignore(); // IMPORTANT: clear buffer for next getline

        cout << "-----\n";
    }

    void displayData()
    {
        cout << "\n--- Book Found ---";
        cout << "\nTitle      : " << title;
        cout << "\nAuthor      : " << author;
```

```

        cout << "\nAccession No : " << accession_num;
        cout << "\nCost      : " << cost;
        cout << "\n-----\n";
    }

    char* getTitle()
    {
        return title;
    }
};

int main()
{
    Book b[5];
    char searchTitle[30];
    int found = 0;

    cout << "Enter Details for 5 Books:\n\n";

    cin.ignore(); // 🔥 VERY IMPORTANT (before first getline)

    for(int i = 0; i < 5; i++)
    {
        cout << "\nBook " << i + 1 << ":\n";
        b[i].readData();
    }

    cout << "\nEnter the Title of the Book to Search: ";
    cin.getline(searchTitle, 30);

    for(int i = 0; i < 5; i++)
    {
        if(strcmp(b[i].getTitle(), searchTitle) == 0)
        {
            b[i].displayData();
            found = 1;
            break;
        }
    }

    if(found == 0)
    {
        cout << "\nBook not found!" << endl;
    }
}

```

```
}  
  
    return 0;  
}
```

Output:

```
Narayan-MacBook-Air:module1 Narayan$ g++ 4-array-of-object.cpp -o  
4-array-of-object
```

```
Narayan-MacBook-Air:module1 Narayan$ ./4-array-of-object
```

```
Enter Details for 5 Books:
```

```
c++ Basics
```

```
Book 1:
```

```
Enter Book Title: Enter Author Name: Bjarne Stroustrup
```

```
Enter Accession Number: 101
```

```
Enter Cost: 405
```

```
-----
```

```
Book 2:
```

```
Enter Book Title: Data Structures
```

```
Enter Author Name: Narayan Kumar
```

```
Enter Accession Number: 102
```

```
Enter Cost: 550
```

```
-----
```

```
Book 3:
```

```
Enter Book Title: Java Programming
```

```
Enter Author Name: Niraj Kumar
```

```
Enter Accession Number: 103
```

```
Enter Cost: 660
```

```
-----
```

```
Book 4:
```

```
Enter Book Title: Python Guide
```

```
Enter Author Name: Niharika Roy
```

```
Enter Accession Number: 104
```

```
Enter Cost: 500
```

```
-----
```

```
Book 5:
```

```
Enter Book Title: Machine Learning
```

Enter Author Name: Saurabh Kumar
Enter Accession Number: 105
Enter Cost: 700

Enter the Title of the Book to Search: Machine Learning

--- Book Found ---

Title : Machine Learning
Author : Saurabh Kumar
Accession No : 105
Cost : 700

Narayan-MacBook-Air:module1 Narayan\$
*/

Practical 7.5: Write a C++ program to demonstrate function overloading by calculating the area of a circle and a rectangle.

```
#include <iostream>

using namespace std;

class AreaCalculator
{
public:

    // Function for Area of Circle
    void calculate_area(float radius)
    {
        float area = 3.14 * radius * radius;

        cout << "\nArea of Circle = " << area << endl;
    }

    // Function for Area of Rectangle
    void calculate_area(float length, float breadth)
    {
        float area = length * breadth;

        cout << "Area of Rectangle = " << area << endl;
    }
};

int main()
{
    AreaCalculator obj;

    float r, l, b;

    cout << "Enter radius of circle: ";
    cin >> r;

    obj.calculate_area(r);

    cout << "\nEnter length and breadth of rectangle: ";
    cin >> l >> b;

    obj.calculate_area(l, b);
```

```
    return 0;  
}
```

Output:

```
Narayan-MacBook-Air:module-2 narayan$ g++ 5-calculating-area.cpp -o  
5-calculating-area
```

```
Narayan-MacBook-Air:module1 Narayan$ ./5-calculating-area
```

```
Enter radius of circle: 5
```

```
Area of Circle = 78.5
```

```
Enter length and breadth of rectangle: 10 4
```

```
Area of Rectangle = 40
```

```
Narayan-MacBook-Air:module1 Narayan$
```

Module 3

Practical 8.1: Write a C++ program to create a class Employee with data members name, employee id, salary and designation. Take input from user and display employee details.

```
#include <iostream>
#include <string>

using namespace std;

class Employee
{
private:
    string name;
    int employee_id;
    float salary;
    string designation;

public:

    // Function to take input
    void getDetails()
    {
        cout << "\nEnter Employee Name: ";
        getline(cin, name);

        cout << "Enter Employee ID: ";
        cin >> employee_id;

        cout << "Enter Salary: ";
        cin >> salary;

        cin.ignore(); // clear buffer before getline

        cout << "Enter Designation: ";
        getline(cin, designation);
    }

    // Function to display data
    void displayDetails()
```



```

{
    cout << "\n--- Employee Profile ---";
    cout << "\nID      : " << employee_id;
    cout << "\nName     : " << name;
    cout << "\nDesignation : " << designation;
    cout << "\nSalary    : " << salary;
    cout << "\n-----\n";
}
};

int main()
{
    Employee emp;

    emp.getDetails();
    emp.displayDetails();

    return 0;
}

```

Output:

```

Narayan-MacBook-Air:module-3 narayan$ g++ 1-employee-class.cpp -o
1-employee-class

```

```

Narayan-MacBook-Air:module-3 narayan$ ./1-employee-class

```

```

Enter Employee Name: Narayan Kumar
Enter Employee ID: 101
Enter Salary: 98000
Enter Designation: Software Engineer (SDE 1)

```

```

--- Employee Profile ---
ID      : 101
Name     : Narayan Kumar
Designation : Software Engineer (SDE 1)
Salary    : 98000
-----

```

```

Narayan-MacBook-Air:module-3 narayan$

```

Practical 3.2: Write a C++ program to find factorial of a number using parameterized constructor.

```
#include <iostream>
```

```
using namespace std;
```

```
class Factorial
```

```
{
```

```
private:
```

```
    int n;
```

```
    long fact;
```

```
public:
```

```
    // Parameterized constructor
```

```
    Factorial(int num)
```

```
{
```

```
    n = num;
```

```
    fact = 1;
```

```
    if (n < 0)
```

```
    {
```

```
        fact = 0;
```

```
    }
```

```
    else
```

```
    {
```

```
        for(int i = 1; i <= n; i++)
```

```
        {
```

```
            fact = fact * i;
```

```
        }
```

```
    }
```

```
    display();
```

```
}
```

```
void display()
```

```
{
```

```
    if(fact == 0)
```

```
    {
```

```
        cout << "\nFactorial of negative numbers is not defined." << endl;
```

```
    }
```

```
    else
```

```
    {
```

```

        cout << "\nFactorial of " << n << " is: " << fact << endl;
    }
}
};

int main()
{
    int val;

    cout << "Enter a number to find factorial: ";
    cin >> val;

    Factorial obj(val);

    return 0;
}

```

Output:

```

Narayan-MacBook-Air:module-3 narayan$ g++
2-factorial-calculation-using-constructor.cpp -o
2-factorial-calculation-using-constructor
Narayan-MacBook-Air:module-3 narayan$
./2-factorial-calculation-using-constructor
Enter a number to find factorial: 5

Factorial of 5 is: 120
Narayan-MacBook-Air:module-3 narayan$

```

Practical 8.3: Write a C++ program to demonstrate operator overloading (=) for copying one 2×2 matrix into another.

```
#include <iostream>

using namespace std;

class Matrix
{
private:
    int mat[2][2];

public:

    // Constructor
    Matrix()
    {
        for(int i = 0; i < 2; i++)
        {
            for(int j = 0; j < 2; j++)
            {
                mat[i][j] = 0;
            }
        }
    }

    // Input function
    void input()
    {
        cout << "Enter 4 elements for 2x2 matrix:\n";

        for(int i = 0; i < 2; i++)
        {
            for(int j = 0; j < 2; j++)
            {
                cin >> mat[i][j];
            }
        }
    }

    // Overloading = operator
    void operator=(const Matrix &m)
    {
```

```

        for(int i = 0; i < 2; i++)
        {
            for(int j = 0; j < 2; j++)
            {
                mat[i][j] = m.mat[i][j];
            }
        }
    }

// Display function
void display()
{
    for(int i = 0; i < 2; i++)
    {
        for(int j = 0; j < 2; j++)
        {
            cout << mat[i][j] << " ";
        }
        cout << endl;
    }
}

};

int main()
{
    Matrix m1, m2;

    cout << "Enter Matrix 1:\n";
    m1.input();

    // Copy using overloaded =
    m2 = m1;

    cout << "\nMatrix 1 (Original):\n";
    m1.display();

    cout << "\nMatrix 2 (Copied):\n";
    m2.display();

    return 0;
}

```

Output:

```
Narayan-MacBook-Air:module-3 narayan$ g++ 3-matrix-copy.cpp -o  
3-matrix-copy
```

```
Narayan-MacBook-Air:module-3 narayan$ ./3-matrix-copy
```

```
Enter Matrix 1:
```

```
Enter 4 elements for 2x2 matrix:
```

```
1 2 3 4
```

```
Matrix 1 (Original):
```

```
1 2
```

```
3 4
```

```
Matrix 2 (Copied):
```

```
1 2
```

```
3 4
```

```
Narayan-MacBook-Air:module-3 narayan$
```

Practical 8.4: Write a C++ program to input a 3×3 matrix and find row-wise sum.

```
#include <iostream>

using namespace std;

int main()
{
    int mat[3][3];
    int i, j, rowSum;

    cout << "Enter elements for a 3x3 matrix:\n";

    for(i = 0; i < 3; i++)
    {
        for(j = 0; j < 3; j++)
        {
            cout << "Element [" << i << "][" << j << "]: ";
            cin >> mat[i][j];
        }
    }

    cout << "\nMatrix with Row-wise Sum:\n";

    for(i = 0; i < 3; i++)
    {
        rowSum = 0;

        for(j = 0; j < 3; j++)
        {
            cout << mat[i][j] << "\t";
            rowSum += mat[i][j];
        }

        cout << "| Sum = " << rowSum << endl;
    }

    return 0;
}
```

Output:

```
Narayan-MacBook-Air:module-3 narayan$ g++ 4-row-wise-sum-of-matrix.cpp -o  
4-row-wise-sum-of-matrix
```

```
Narayan-MacBook-Air:module-3 narayan$ ./4-row-wise-sum-of-matrix
```

```
Enter elements for a 3x3 matrix:
```

```
Element [0][0]: 1
```

```
Element [0][1]: 2
```

```
Element [0][2]: 3
```

```
Element [1][0]: 4
```

```
Element [1][1]: 5
```

```
Element [1][2]: 6
```

```
Element [2][0]: 7
```

```
Element [2][1]: 8
```

```
Element [2][2]: 9
```

```
Matrix with Row-wise Sum:
```

```
1    2    3    | Sum = 6
```

```
4    5    6    | Sum = 15
```

```
7    8    9    | Sum = 24
```

```
Narayan-MacBook-Air:module-3 narayan$
```

```
*/
```


Practical 8.5: Write a C++ program to sort an array in descending order.

```
#include <iostream>

using namespace std;

int main()
{
    int temp;
    int a[10] = {10, 8, 7, 3, 11, 12, 7, 8, 9, 1};
    int i, j;

    // Sorting in descending order
    for(i = 0; i < 10; i++)
    {
        for(j = i + 1; j < 10; j++)
        {
            if(a[i] < a[j])
            {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }

    cout << "Sorted array in descending order:\n";

    for(i = 0; i < 10; i++)
    {
        cout << a[i] << endl;
    }

    return 0;
}
```

Output:

```
Narayan-MacBook-Air:module-3 narayan$ g++ 5-sorting-of-number.cpp -o  
5-sorting-of-number
```

```
Narayan-MacBook-Air:module-3 narayan$ ./5-sorting-of-number
```

```
Sorted array in descending order:
```

```
12
```

```
11
```

```
10
```

```
9
```

```
8
```

```
8
```

```
7
```

```
7
```

```
3
```

```
1
```

```
Narayan-MacBook-Air:module-3 narayan$
```

Practical 8.6: Write a C++ program to find the transpose of a 3×3 matrix.

```
#include <iostream>

using namespace std;

#define ROW 3
#define COL 3

int main()
{
    int A[ROW][COL];
    int i, j, temp;

    cout << "Enter 9 elements of original matrix:\n";

    for(i = 0; i < ROW; i++)
    {
        for(j = 0; j < COL; j++)
        {
            cin >> A[i][j];
        }
    }

    cout << "\nOriginal Matrix:\n";

    for(i = 0; i < ROW; i++)
    {
        for(j = 0; j < COL; j++)
        {
            cout << A[i][j] << "t";
        }
        cout << endl;
    }

    // Transpose in-place
    for(i = 0; i < ROW; i++)
    {
        for(j = i + 1; j < COL; j++)
        {
            temp = A[i][j];
            A[i][j] = A[j][i];
            A[j][i] = temp;
        }
    }
}
```

```

    }
}

cout << "\nTranspose of Matrix:\n";

for(i = 0; i < ROW; i++)
{
    for(j = 0; j < COL; j++)
    {
        cout << A[i][j] << "t";
    }
    cout << endl;
}

return 0;
}

```

Output:

```

Narayan-MacBook-Air:module-3 narayan$ g++ 6-transpose-of-numbers.cpp -o
6-transpose-of-numbers
Narayan-MacBook-Air:module-3 narayan$ ./6-transpose-of-numbers
Enter 9 elements of original matrix:
1 2 3 4 5 6 7 8 9

```

Original Matrix:

1	2	3
4	5	6
7	8	9

Transpose of Matrix:

1	4	7
2	5	8
3	6	9

```

Narayan-MacBook-Air:module-3 narayan$

```

Module 4

Practical 9.1: Write a C++ program to add two numbers using a function and display the result.

```
#include <iostream>
using namespace std;

long int sum(long int num1, long int num2)
{
    return num1 + num2;
}

int main()
{
    long int a, b;

    cout << "Enter two numbers: ";
    cin >> a >> b;

    long int res = sum(a, b);

    cout << "The Result = " << res << endl;

    return 0;
}
```

Output:

```
Narayan-MacBook-Air:module-4 narayan$ g++ 1-add-function-demo.cpp -o
1-add-function-demo
Narayan-MacBook-Air:module-4 narayan$ ./1-add-function-demo
Enter two numbers: 44 22
The Result = 66
Narayan-MacBook-Air:module-4 narayan$
```

Practical 9.2: Write a C++ program to write characters into a file and then read and display them using file handling.

```
#include <iostream>
#include <fstream>

using namespace std;

int main() {
    ofstream fout;
    ifstream fin;
    char ch;
    int n;

    // Write characters to file
    fout.open("chars.txt");
    if (!fout) {
        cout << "Error opening file for writing!";
        return 0;
    }

    cout << "How many characters do you want to write? ";
    cin >> n;

    cout << "Enter " << n << " characters:\n";
    for (int i = 0; i < n; i++) {
        cin >> ch;
        fout.put(ch);
    }

    fout.close();

    // Read characters from file
    fin.open("chars.txt");
    if (!fin) {
        cout << "Error opening file for reading!";
        return 0;
    }

    cout << "\nCharacters stored in the file are:\n";

    while (fin.get(ch)) {
        cout << ch << " ";
    }
}
```

```
}  
  
fin.close();  
  
return 0;  
}
```

Output:

```
Narayan-MacBook-Air:module-4 narayan$ g++  
2-character-reading-and-writing-to-file.cpp -o  
2-character-reading-and-writing-to-file  
Narayan-MacBook-Air:module-4 narayan$  
./2-character-reading-and-writing-to-file  
How many characters do you want to write? 5  
Enter 5 characters:  
a b c d e
```

```
Characters stored in the file are:  
Narayan-MacBook-Air:module-4 narayan$
```

Practical 9.3: Write a C++ program to demonstrate constructors, destructor, and member functions using a class.

```
#include <iostream>
using namespace std;

class A {
private:
    int x;
    float y;

public:
    A();           // default constructor
    A(int);        // parameterized constructor 1
    A(int, float); // parameterized constructor 2
    ~A();          // destructor

    void setValue(int, float);
    int getValueInt();
    float getValueFloat();
};

// Destructor
A::~~A() {
    cout << "\nDestructor Invoked!!\n";
}

// Default constructor
A::A() {
    x = 10;
    y = 45.5;
    cout << "Default constructor invoked\n";
}

// Parameterized constructor 1
A::A(int a) {
    x = a;
    y = 0;
}

// Parameterized constructor 2
A::A(int a, float b) {
    x = a;
```



```

    y = b;
}

void A::setValue(int a, float b) {
    x = a;
    y = b;
}

int A::getValueInt() {
    return x;
}

float A::getValueFloat() {
    return y;
}

int main() {
    A obj1; // default constructor
    cout << "\nValue of obj1 (int): " << obj1.getValueInt() << "\n";

    A obj2(99);
    cout << "Value of obj2 (int): " << obj2.getValueInt() << "\n";

    A obj3(88, 67.8);
    cout << "Value of obj3 (int): " << obj3.getValueInt() << "\n";
    cout << "Value of obj3 (float): " << obj3.getValueFloat() << "\n";

    obj1.setValue(50, 56.8);
    cout << "Updated obj1 (int): " << obj1.getValueInt() << "\n";

    return 0;
}

```

Output:

```
Narayan-MacBook-Air:module-4 narayan$ g++  
3-constructors-and-polymorphism.cpp -o 3-constructors-and-polymorphism  
Narayan-MacBook-Air:module-4 narayan$ ./3-constructors-and-polymorphism  
Default constructor invoked
```

```
Value of obj1 (int): 10  
Value of obj2 (int): 99  
Value of obj3 (int): 88  
Value of obj3 (float): 67.8  
Updated obj1 (int): 50
```

```
Destructor Invoked!!
```

```
Destructor Invoked!!
```

```
Destructor Invoked!!  
Narayan-MacBook-Air:module-4 narayan$
```

Practical 9.4: Write a C++ program to copy contents of one file into another file using file handling.

```
#include <iostream>
#include <fstream>

using namespace std;

int main() {
    char sourceFile[100], targetFile[100], ch;

    cout << "Enter the name of the source file (e.g., DATA.txt): ";
    cin >> sourceFile;

    cout << "Enter the name of the target file: ";
    cin >> targetFile;

    // Open source file
    ifstream fin(sourceFile);
    if (!fin) {
        cout << "Error: Source file not found!";
        return 1;
    }

    // Open target file
    ofstream fout(targetFile);
    if (!fout) {
        cout << "Error: Could not create target file!";
        fin.close();
        return 1;
    }

    // Copy file character by character
    while (fin.get(ch)) {
        fout.put(ch);
    }

    cout << "\nFile contents copied successfully.\n";

    fin.close();
    fout.close();

    return 0;
}
```

```
}
```

Output:

```
Narayan-MacBook-Air:module-4 narayan$ g++ 4-copy-file-content.cpp -o  
4-copy-file-content
```

```
Narayan-MacBook-Air:module-4 narayan$ ./4-copy-file-content
```

```
Enter the name of the source file (e.g., DATA.txt): input.txt
```

```
Enter the name of the target file: output.txt
```

```
File contents copied successfully.
```

```
Narayan-MacBook-Air:module-4 narayan$
```

Practical 9.5: Write a C++ program to count the number of words in a file using file handling.

```
#include <iostream>
#include <fstream>
#include <cctype>

using namespace std;

int main() {
    char filename[100], ch;
    int wordCount = 0;
    int inWord = 0;

    cout << "Enter the filename to open: ";
    cin >> filename;

    ifstream fin(filename);

    if (!fin) {
        cout << "Error: File could not be opened!";
        return 1;
    }

    while (fin.get(ch)) {
        if (isspace(static_cast<unsigned char>(ch))) {
            inWord = 0;
        }
        else if (inWord == 0) {
            inWord = 1;
            wordCount++;
        }
    }

    cout << "\nTotal number of words in the file: " << wordCount << endl;

    fin.close();
    return 0;
}
```

Output:

```
Narayan-MacBook-Air:module-4 narayan$ g++ 5-counting-words.cpp -o  
5-counting-words
```

```
Narayan-MacBook-Air:module-4 narayan$ ./5-counting-words
```

```
Enter the filename to open: input.txt
```

```
Total number of words in the file: 10
```

```
Narayan-MacBook-Air:module-4 narayan$
```

Practical 9.6: Write a C++ program to store even numbers in one file and odd numbers in another file and display their contents.

```
#include <iostream>
#include <fstream>

using namespace std;

int main() {

    ofstream ofs1("even.txt");
    ofstream ofs2("odd.txt");

    // Write even and odd numbers
    for (int i = 0; i <= 10; i++) {
        if (i % 2 == 0) {
            ofs1 << i << endl;
        } else {
            ofs2 << i << endl;
        }
    }

    ofs1.close();
    ofs2.close();

    cout << "The content is written to files successfully.\n\n";

    // Read even file
    ifstream ifs1("even.txt");
    int num;

    cout << "The content of even file is:\n";
    while (ifs1 >> num) {
        cout << num << endl;
    }
    ifs1.close();

    // Read odd file
    ifstream ifs2("odd.txt");

    cout << "\nThe content of odd file is:\n";
    while (ifs2 >> num) {
        cout << num << endl;
    }
}
```

```
}  
ifs2.close();  
  
return 0;  
}
```

Output:

```
Narayan-MacBook-Air:module-4 narayan$ g++ 6-file-reading-and-writing.cpp -o  
6-file-reading-and-writing  
Narayan-MacBook-Air:module-4 narayan$ ./6-file-reading-and-writing  
The content is written to files successfully.
```

The content of even file is:

```
0  
2  
4  
6  
8  
10
```

The content of odd file is:

```
1  
3  
5  
7  
9
```

```
Narayan-MacBook-Air:module-4 narayan$
```


Practical 9.7: Write a C++ program to demonstrate structure, class, and pointer to object.

```
#include <iostream>
#include <cstring>

using namespace std;

struct Employee {
    int emp_no;
    char name[20];

    int get_emp_no() {
        return emp_no;
    }
};

class Student {
    int roll;
    char name[20];

public:
    Student(int r) {
        roll = r;
    }

    void set_values(int r, const char* n) {
        roll = r;
        strcpy(name, n);
    }

    int get_roll() {
        return roll;
    }

    void set_name() {
        cout << "Enter student name: ";
        cin >> name; // safer than gets()
    }

    char* get_name() {
        return name;
    }
};
```

```

};

int main() {
    Employee e;
    Student s(20);

    e.emp_no = 100;

    cout << "Employee No: " << e.get_emp_no() << endl;
    cout << "Roll No: " << s.get_roll() << endl;

    s.set_name();
    cout << "Student Name: " << s.get_name() << endl;

    s.set_values(8, "Akash");
    cout << "Roll No: " << s.get_roll() << endl;
    cout << "Student Name: " << s.get_name() << endl;

    cout << "\nUse of Pointer to object" << endl;

    Student obj(0);    // create object first
    Student *ptr = &obj; // pointer points to object

    ptr->set_name();
    cout << "Name of the Student: " << ptr->get_name() << endl;

    return 0;
}

```

Output:

```
Narayan-MacBook-Air:module-4 narayan$ g++ 7-pointer-to-object.cpp -o
7-pointer-to-object
Narayan-MacBook-Air:module-4 narayan$ ./7-pointer-to-object
Employee No: 100
Roll No: 20
Enter student name: Narayan
Student Name: Narayan
Roll No: 8
Student Name: Akash

Use of Pointer to object
Enter student name: Neha
Name of the Student: Neha
Narayan-MacBook-Air:module-4 narayan$
*/
```

Practical 9.8: Write a C++ program using switch-case to select gender based on input character (M/F).

```
#include <iostream>

using namespace std;

int main() {
    char ch;

    cout << "Enter a Character M or F to Select Gender Option: ";
    cin >> ch;

    switch (ch) {
        case 'M':
        case 'm':
            cout << "Gender Selection is Male" << endl;
            break;

        case 'F':
        case 'f':
            cout << "Gender Selection is Female" << endl;
            break;

        default:
            cout << "Invalid Selection" << endl;
    }

    return 0;
}
```

Output:

```
Narayan-MacBook-Air:module-4 be g++ 8-switch-for-characters.cpp
-o 8-switch-for-characters
Narayan-MacBook-Air:module-4 narayan$ ./8-switch-for-characters
Enter a Character M or F to Select Gender Option: M
Gender Selection is Male
Narayan-MacBook-Air:module-4 narayan$ ./8-switch-for-characters
Enter a Character M or F to Select Gender Option: F
Gender Selection is Female
Narayan-MacBook-Air:module-4 narayan$
```